

CS 15: Data Structures

Welcome!

Some Logistics

Instructor: Chris Magnano (he/him)

- ▶ pronunciation: “kris mon-yawn-oh”

Email: christopher.magnano@tufts.edu

- ▶ But use Piazza

Office Hours:

- ▶ Tuesdays 3:00-4:00pm: <https://tufts.zoom.us/my/cmagnano>
- ▶ Wednesdays 1:30-2:30pm in JCC 449 (my office)

Course Website: www.cs.tufts.edu/comp/15

- ▶ Contains all HWs, labs, slides, and more

My Background



- Research background is in network analysis, computational biology, and exploratory analysis.
- I've also been a software developer for Epic Systems (the healthcare software not the videogame company).
- I entered college as a music and biology double major, and didn't take a CS course until my sophomore year.

My Responsibilities

- I am responsible for facilitating a classroom culture based on trust and respect. You are here to learn and as such **I will never mock you or make you feel ashamed for a question you ask or information you do not know.**
- I am responsible for treating you with the respect and value you deserve as a human being, and my goal is to make my classroom an environment set up for your success.
- I am responsible for creating assignments which primarily aid your learning and are fair assessments. I am responsible for making my expectations about these assignments clear and grading them with productive feedback in a timely manner.
- I am responsible for coming to class on time, prepared, and engaged. I am also responsible for making myself available outside of class time for additional discussion and help.

Your Responsibilities

- You are responsible for the way you treat your peers. This involves treating each other with kindness and taking responsibility for your own words and actions.
- You are responsible for attending class regularly. This includes being prepared for class and engaging in in-class activities.
- You are responsible for completing assignments in a timely manner. It is important for your learning, my schedule, and fairness for other members of the class that you complete assignments on time (more details in a bit).
- **You are ultimately responsible for your own learning.** This means engaging in the course and getting help when you are confused.
- You are responsible for maintaining academic honesty in all your work.

Your TAs

- They are incredible, and they're here to help!
- Office hours:
 - ▶ Begin on Wednesday, January 21
 - ▶ Take place in JCC 3rd floor common area next to elevators
 - ▶ Details will be posted on Piazza

Lecture Recordings

Will be somewhere in this list.

Home

Echo360

Discussions

Grades

People

Syllabus

Course Evaluations

Zoom

Reading Lists

Media Gallery

- All lectures are being recorded!
- Recordings only shared with enrolled students
- You can find them on Canvas, under “Echo360”
 - ▶ <https://canvas.tufts.edu>
 - ▶ Note that this is the only thing Canvas is used for in this class.

Course Forum

- We will use Piazza as our course forum
- You should have been enrolled already
 - ▶ If not, email me
- You must join and keep up with Piazza
 - ▶ We will regularly post announcements here
- Use Piazza to:
 - ▶ Post publicly: Common questions which pertain to all students
 - Never share solutions!!!
 - Always search first to see if question has been asked before
 - ▶ Post privately: Questions containing code, or private info

Course Website

- <https://www.cs.tufts.edu/comp/15/>
- Schedule contains all slides, HWs, labs, and readings
- Reference page includes helpful... references
 - C++ help, style guide, CS 11 lectures, ...
- Admin page covers course policy
 - You must read the admin page!

Course Policies

Can I get extensions on HWs?

How is my final grade calculated?

How do I request a regrade?

How can I view my HW autograder score?

What extra support can I access?

Read the course admin page!

Homework

- See course webpage for HW schedule
 - ▶ 2 HWs
 - ▶ 4 projects
 - ▶ Projects have multiple stages
- HWs: 7 days
- Projects: 14 days
- **You should always start early**

Labs

- Every Tuesday (you should already be signed up)
 - ▶ Note: We are combining some lab sessions
- We take attendance: You must come to lab!
 - ▶ You may miss up to 2 lab sessions and receive full credit
 - *But you still must submit work for these labs*
- Labs are low-stress: You will receive full credit for submitting a *good-faith* effort
- Labs are due at the latest by Friday at 6pm
 - ▶ But you should submit your work at the end of lab on Tuesday (can always resubmit later)
- Some labs have a pre-lab. Do these!

Academic Integrity

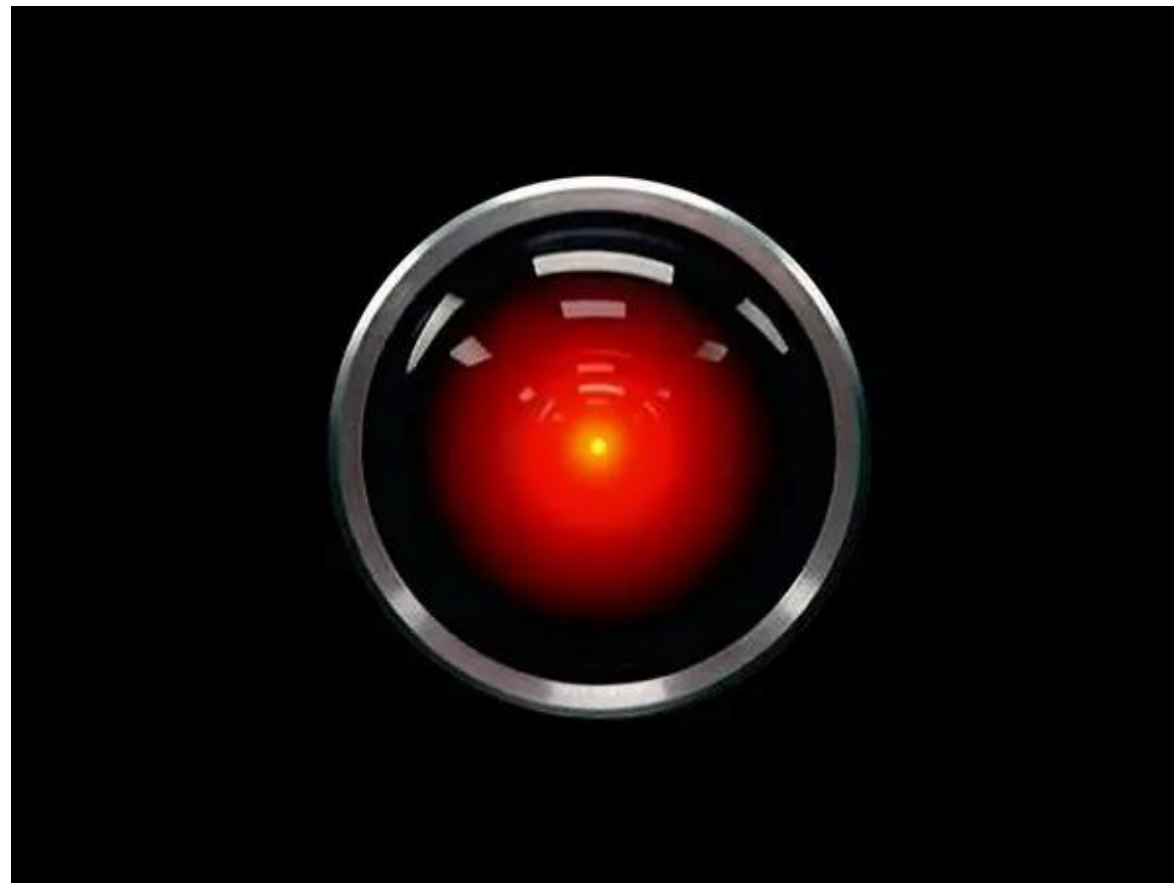
- Read our policy on the course admin page
- Students may discuss HWs, projects *at a high level*
 - i.e., using natural languages and drawings
- Students may not show code to anyone other than course staff
- We rely on automated systems for detecting plagiarism
 - We compare submissions with current and previous semesters
- Any suspected violation will be reported to Dean of Student Affairs office for an investigation

Artificial Intelligence

- We have a new resource: the CS15 AI Chatbot
 - ▶ Access at: <https://www.cs.tufts.edu/comp/15/chatbot/>
 - ▶ Or, install the “CS15 Tutor” extension in VSCode
- This is an LLM-backed chatbot that incorporates our course materials: lectures, HWs, projects, and more
- It does save conversations but does not save identifying information (*but* your conversation could be enough to identify you)
- Please be aware: it can get information wrong!!!
- Huge thanks to TAs Valentina Henao Giraldo and Nahuel Gomez Ruiz for this effort!

Academic Integrity & AI

No using other generative
AI models: ChatGPT, Github
Copilot, Cursor, etc.!



Textbook Readings

- We regularly post readings from the online textbook *Data Structures and Algorithm Analysis* by C.A. Shaffer
- Textbook readings are *optional*
- However, you will usually find them to be a useful supplement to lectures

Using C++, Terminal

- This class assumes you have the basics down
 - Everything covered in CS 11
- We have some resources to support you!
 - See course reference page
 - It includes some of my recorded lectures from CS 11
- (Optional) Lab 0 introduces students to writing and submitting programs

Keys to Success

1. Start early.
2. Think before you code.
3. Write a little, test a little.
4. Ask us for help!

Learning in this Class

- Computer science *can be challenging*
- ...but it is not beyond your reach!

```

* @var boolean
*/
define('PSI_INTERNAL_XML', false);

if (version_compare("5.2", PHP_VERSION, ">")) {
    die("PHP 5.2 or greater is required!!!");
}

if (!extension_loaded("pcre")) {
    die("phpSysInfo requires the pcre extension to php in order to work properly.");
}

require_once APP_ROOT.'/includes/autoloader.inc.php';

// Load configuration
require_once APP_ROOT.'/config.php';

if (!defined('PSI_CONFIG_FILE')) || !defined('PSI_DEBUG')) {
    $tpl = new Template("/templates/html/error_config.html");
    echo $tpl->fetch();
    die();
}

// javascript
1: strtolower(
```

If you have prior background in CS:
Please be respectful of others who are learning.

TODOs for Next Week

- Read course admin page
- Browse course reference page
 - Review any materials you wish
- Optional:
 - Lab 0: Basics of terminal, working remotely
 - Read posting from Shaffer textbook

Data Structures



data structure **noun**

Definition of *data structure*

: any of various methods or formats (such as an array, file, or record) for organizing data in a computer

Why Data Structures?

Why Data Structures?

There are two big reasons why this subject is important:

1. Abstraction
2. Performance

What is abstraction?

- In CS, *abstraction* is the hiding of low-level details to focus on details of greater importance
 - It makes complexity more manageable
- Abstraction is *fundamental* to computers and CS
 - Without it, we could not build complex systems

Abstraction in Computers

- When we write programs, we rely on many layers of abstraction
- Each layer *assumes* certain functionality from the previous layer. Doesn't care about low-level details!

High-Level Language

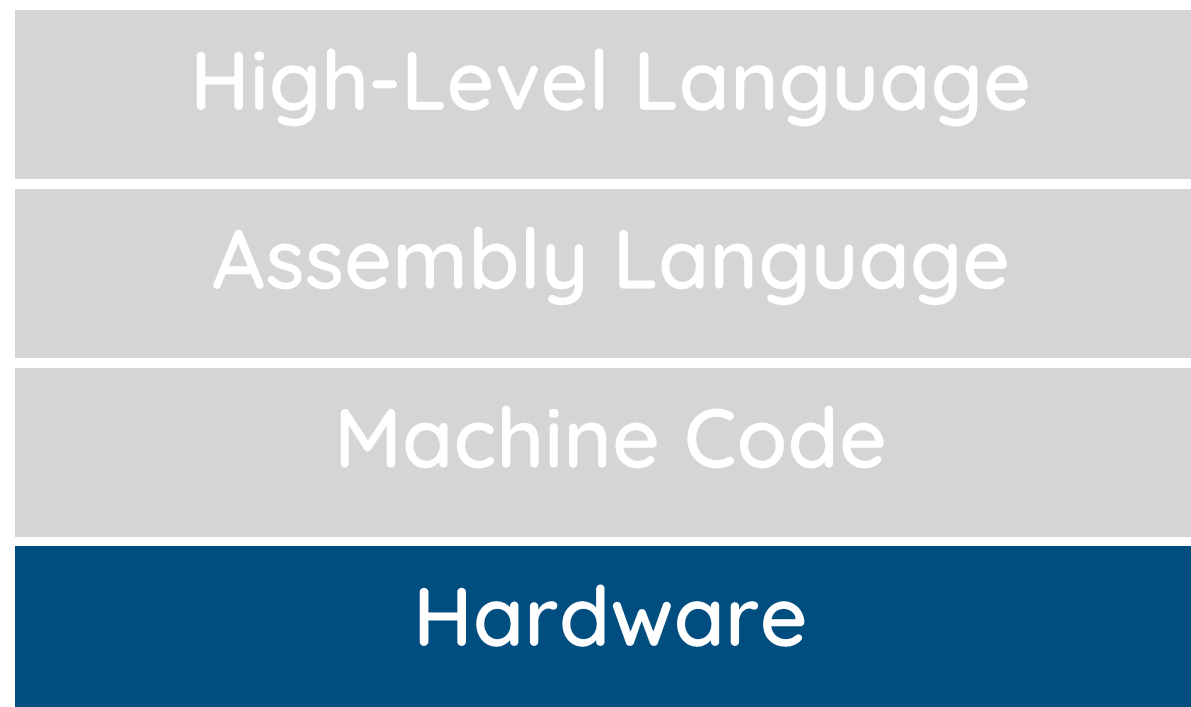
Assembly Language

Machine Code

Hardware

Abstraction in Computers

- Hardware: the physical components of computers
- In fact, there are many layers of abstraction *within* this layer:



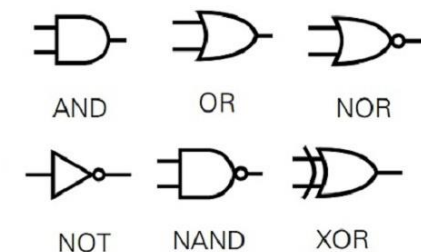
Motherboard



Integrated Circuits (chips)



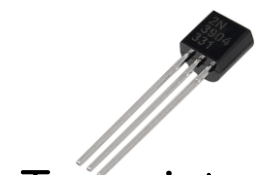
Logic Gates



Circuits



Wires



Transistors

Abstraction in Computers

- Machine Code: instructions, expressed in binary numbers, that cause the CPU to perform a specific task

High-Level Language

Assembly Language

Machine Code

Hardware

```
00000000: 01001101 01011010 10010000 00000000 00000011 00000000 MZ....
00000006: 00000000 00000000 00000100 00000000 00000000 00000000 .....
0000000c: 11111111 11111111 00000000 00000000 10111000 00000000 .....
00000012: 00000000 00000000 00000000 00000000 00000000 00000000 .....
00000018: 01000000 00000000 00000000 00000000 00000000 00000000 @.....
0000001e: 00000000 00000000 00000000 00000000 00000000 00000000 .....
00000024: 00000000 00000000 00000000 00000000 00000000 00000000 .....
0000002a: 00000000 00000000 00000000 00000000 00000000 00000000 .....
00000030: 00000000 00000000 00000000 00000000 00000000 00000000 .....
00000036: 00000000 00000000 00000000 00000000 00000000 00000000 .....
0000003c: 10000000 00000000 00000000 00000000 00001110 00011111 .....
00000042: 10111010 00001110 00000000 10110100 00001001 11001101 .....
00000048: 00100001 10111000 00000001 01001100 11001101 00100001 !..L.!
0000004e: 01010100 01101000 01101001 01110011 00100000 01110000 This p
00000054: 01110010 01101111 01100111 01110010 01100001 01101101 rogram
0000005a: 00100000 01100011 01100001 01101110 01101110 01101111 canno
00000060: 01110100 00100000 01100010 01100101 00100000 01110010 t be r
00000066: 01110101 01101110 00100000 01101001 01101110 00100000 un in
0000006c: 01000100 01001111 01010011 00100000 01101101 01101111 DOS mo
00000072: 01100100 01100101 00101110 00001101 00001101 00001010 de...
00000078: 00100100 00000000 00000000 00000000 00000000 00000000 $. ....
0000007e: 00000000 00000000 01010000 01000101 00000000 00000000 ..PE..
```

Abstraction in Computers

- Assembly language: *slightly* higher-level than machine code
 - Instructions are more English-like
 - Some instructions map to multiple machine code instructions

High-Level Language

Assembly Language

Machine Code

Hardware

```
movl $0xFF001122, %eax
addl %ecx, %edx
xorl %esi, %esi
pushl %ebx
movl 4(%esp), %ebx
leal (%eax,%ecx,2), %esi
cmpl %eax, %ebx
jnae foo
retl
```


Abstraction in Computers

- High-level language: C++, Java, Python, Ruby, ...

High-Level Language

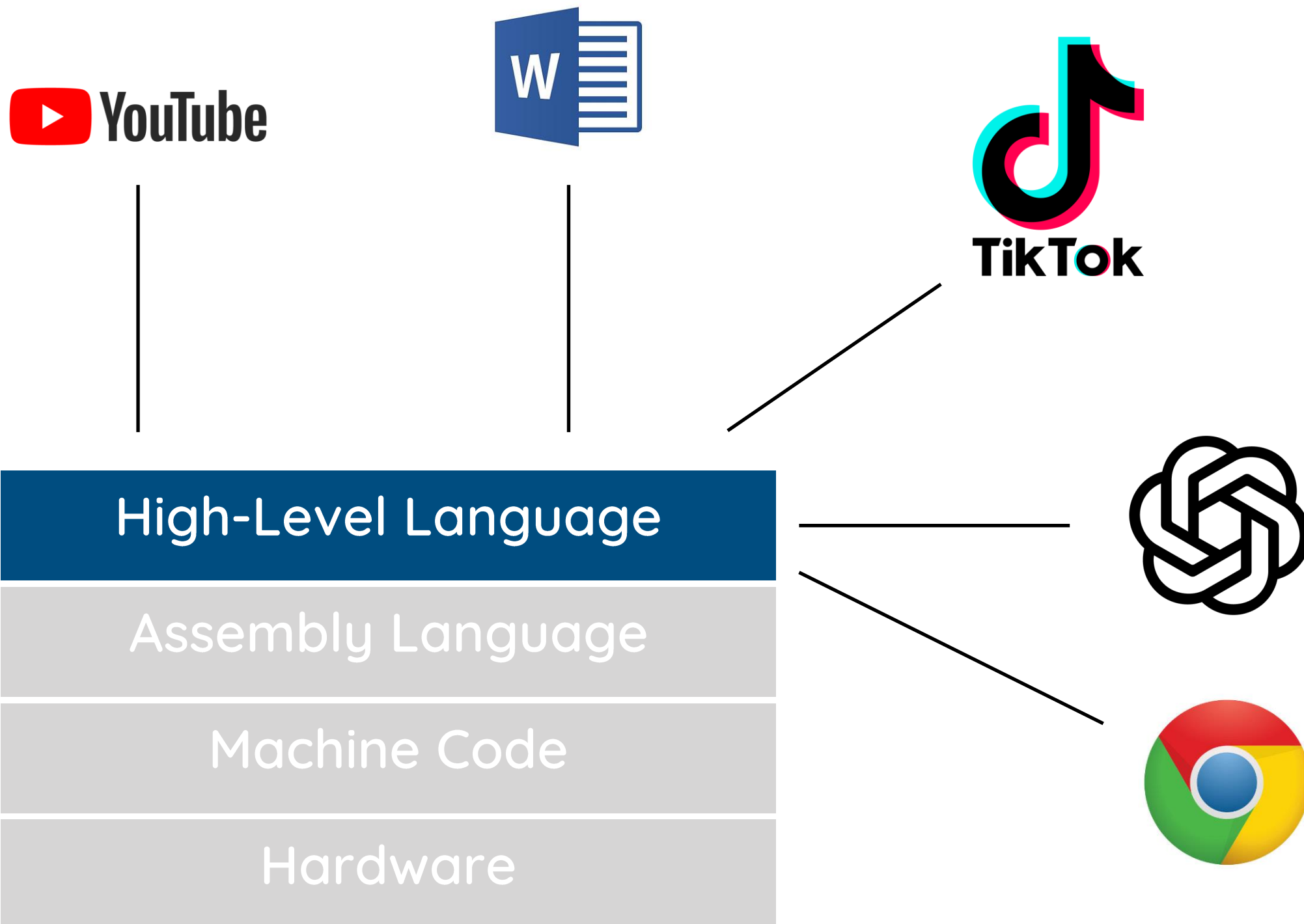
Assembly Language

Machine Code

Hardware

```
332 ~mvThreadPool() { m_done = true; }
333
334 static const char* getVersion() { return "v0.2"; }
335
336 template<typename F, typename ...Args>
337 std::future<typename std::invoke_result<F, Args...>::type> submit(F f)
338 {
339     typedef typename std::invoke_result<F, Args...>::type result_type;
340     std::packaged_task<result_type()> task(std::move(f));
341     std::future<result_type> res(task.get_future());
342     if (m_local_work_queue)
343         m_local_work_queue->push(std::move(task));
344     else
345         m_pool_work_queue.push(std::move(task));
346
347     return res;
348 }
349
350 private:
```

Abstraction in Computers



The Power of Abstraction

- When operating at a higher layer, we don't need to care about the lower-level details
- When I program in C++, I don't care about circuits or transistors
- When I watch Netflix, I'm not worried about the code it comprises

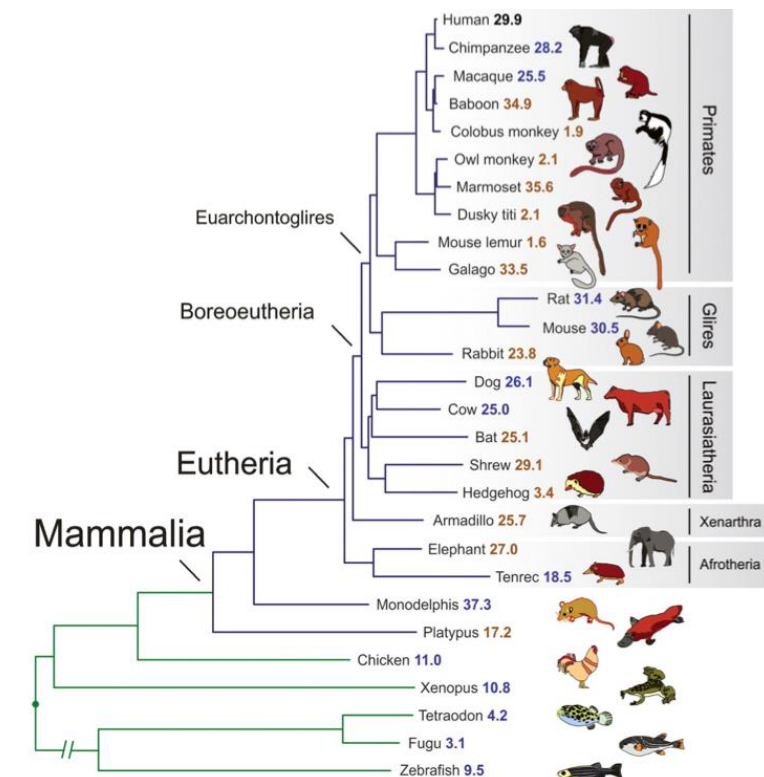
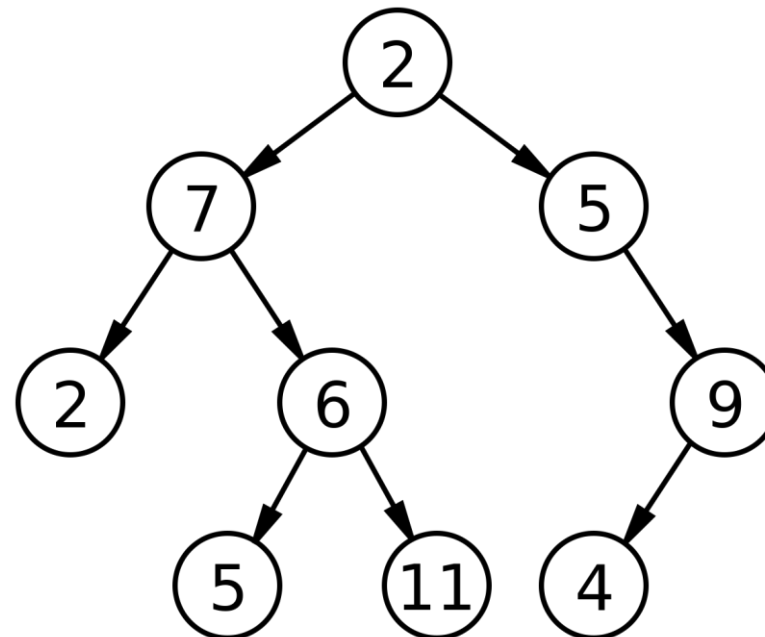
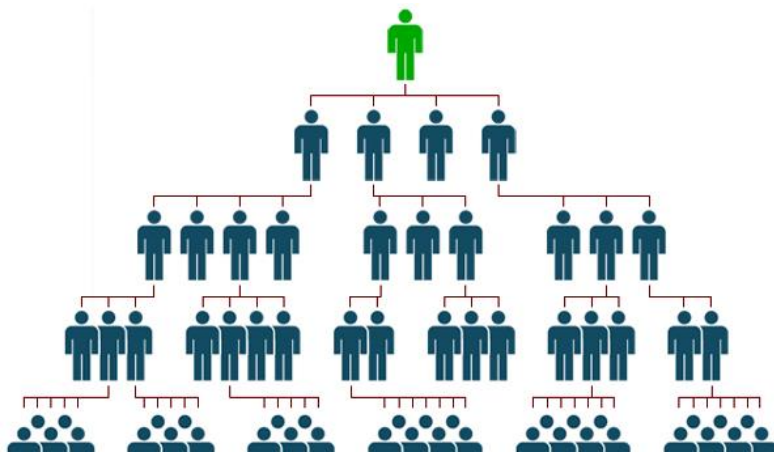


What About Data Structures?

- In theory, *every single program* could be written using only arrays
- But this would get *extremely* complex, tedious
- Different data structures provide different abstractions
 - Which you should use depends on your goal!

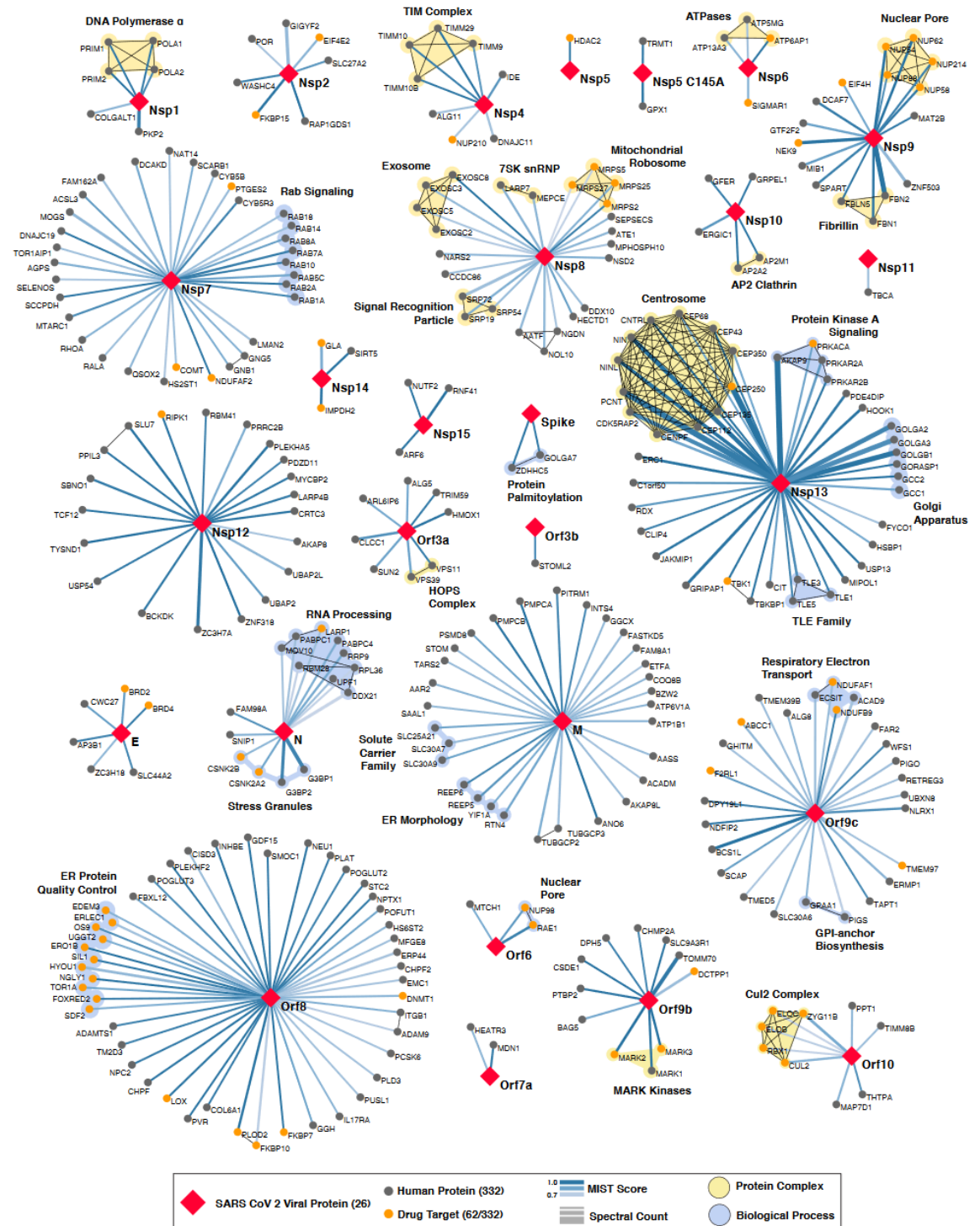
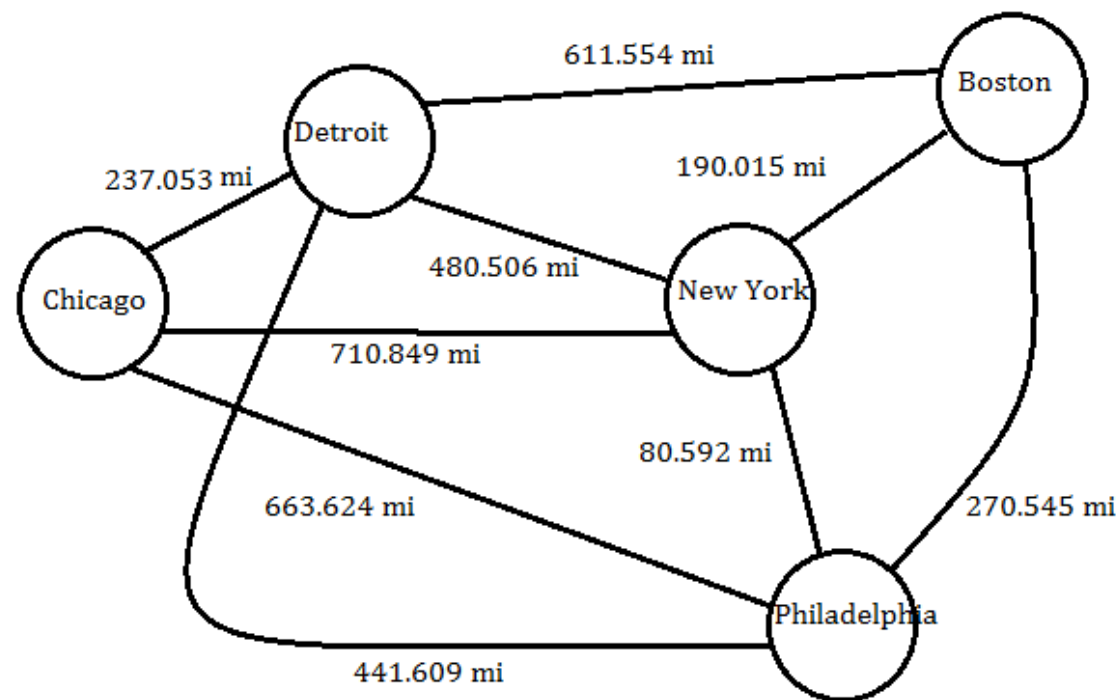
Data Structures and Abstractions

- What if you want to represent hierarchical relationships among entities?
- Use *trees*!



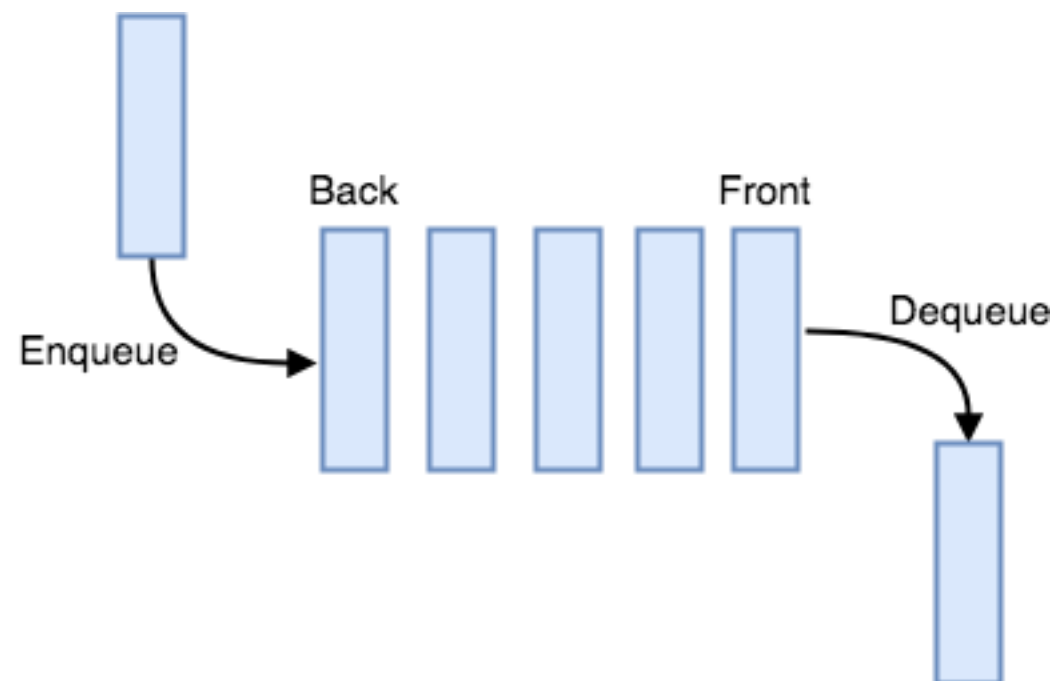
Data Structures and Abstractions

- Representing relationships or distances?
- Use *graphs*



Data Structures and Abstractions

- Representing an ordered line?
- Use *queues*



Why Data Structures?

There are two big reasons why this subject is important:

1. Abstraction
2. Performance

Performance

- Different data structures are associated with various costs/benefits in *time* and *space*
- These are *crucial* considerations, especially as programs grow larger, handle more data
- In CS 15, we will learn to *formally* classify and compare costs in time/space associated with data structures

CS 15

- This course will:
 - ▶ Equip you with fundamental tools—data structures—for writing programs
 - ▶ Teach you how to pick *which* data structure is appropriate
 - ▶ Train you in writing *medium-sized* programs

This is one of the *most useful* courses for employment, interviews